

SoundMatch

1. Project Overview:

SoundMatch processes raw audio files to extract meaningful features that describe how a song sounds (such as frequency patterns, rhythm, timbre and texture) by converting them into numerical representations a model can use. Each song is summarized as a feature vector capturing its overall sonic character. These vectors are indexed for fast nearest-neighbor search, allowing the system to find sonically similar tracks based purely on audio content rather than genre tags or metadata. Given a query song, SoundMatch extracts its features using the same pipeline and returns the most acoustically similar tracks from its dataset, as track IDs or song files, ranked by similarity. This enables discovery across traditional genre boundaries, surfacing tracks that *sound* alike even when their labels differ.

2. Type of data:

Mel-frequency Cepstral Coefficients (MFCC): Capture the timbre (texture and character of sound). They summarize how the sound is shaped across frequencies. The mean and standard deviation are used to create a fixed-size representation.

Mel Spectrogram: Represents the energy of frequencies over time using a scale similar to human hearing. It gives a more detailed view of the sound than MFCC. The mean and standard deviation are used for summarization.

Tempo: The speed of the music, measured in beats per minute (BPM). Usually a single value per track (mean is sufficient; standard deviation is typically not needed).

Onset: Represents the moments when new sounds or notes begin (e.g., beats or attacks). Often summarized as onset density (number of onsets relative to duration) rather than raw onset times.

To avoid excessive memory consumption, extracted features are streamed directly to a CSV file instead of being accumulated in memory.

3. Model:

SoundMatch uses a k-Nearest Neighbors model to find sonically similar songs. During training, each song's 156 audio features (MFCCs, mel spectrogram statistics, tempo, and onset density) are standardized so that no single feature dominates the distance calculations. The scaled features are then indexed by NearestNeighbors, which builds an internal tree structure for fast lookup. The fitted model (including the scaler, the search index, and the song IDs) is saved to a single joblib file so it can be reused without retraining.

When a new song is provided, it passes through the same feature extraction pipeline used during training. The resulting feature vector is then handed to the search index, which returns the 3 nearest training songs based on Euclidean distance. The output is a ranked list of song IDs along with their distances, where smaller distances indicate greater sonic similarity.

Appendix:

Inspired by: https://www.youtube.com/watch?v=V5Psciq3T_g&t=331s

Data used: <https://github.com/mdeff/fma> (the small data set)

AI disclosure:

This project was developed with the assistance of Claude, a large language model by Anthropic. AI was used to clarify technical concepts (such as how audio features and clustering algorithms work), review and debug code and refine the wording of the documentation and README. All design decisions, implementation choices and final outputs were reviewed and approved by the author. The project structure, feature engineering choices, and overall direction are the author's own work.